

Greedy and Dynamic Programming (Program)

Efficient Algorithms until now!

- ◆ We have seen problems which have immediate (somewhat trivial) polynomial time algorithms, e.g., $O(n^2)$, $O(n^3)$, etc...
- ◆ We studied how to get them better ... e.g., $O(n \log n)$...
 - ◆ For that, we have used **Divide & Conquer** and other tricks.

What lies beyond?

- ◆ There are many situations, when it's not even clear what's the obvious polynomial time algorithm.
- ◆ Brute-force type algorithms could take exponential time.
- ◆ What is the solution?

What lies beyond?

Greedy

(Take the best you can)

Dynamic Programming

(Recurse & Reuse)

What lies beyond?

Greedy

(Take the best you can)

Dynamic Programming

(Recurse & Reuse)

Use What gives
better deal!

Charting the path!

- ◆ Greedy algorithms
 - ◆ Local optimality.
 - ◆ Near-sighted.
 - ◆ Doesn't care about long-run.

Charting the path!

- ◆ Greedy algorithms
 - ◆ Local optimality.
 - ◆ Near-sighted.
 - ◆ Doesn't care about long-run.

Proof of Correctness is important.

It's not clear in many cases...

Charting the path!

- ◆ Greedy algorithms
 - ◆ Local optimality.
 - ◆ Near-sighted.
 - ◆ Doesn't care about long-run.
- ◆ Dynamic Programming
 - ◆ Recursion with better memory management.
 - ◆ Or, a systematic approach to 'search' through all possible solutions.

Proof of Correctness is important.

It's not clear in many cases...

Charting the path!

- ◆ Greedy algorithms
 - ◆ Local optimality.
 - ◆ Near-sighted.
 - ◆ Doesn't care about long-run.
- ◆ Dynamic Programming
 - ◆ Recursion with better memory management.
 - ◆ Or, a systematic approach to 'search' through all possible solutions.

Proof of Correctness is important.
It's not clear in many cases...

Difference between Exponential and
Polynomial is subtle...

Problem 1

- ◆ Given a basket of apples, you want to choose maximum Five of them, and the goal is to maximise the weight.
- ◆ Greedy approach works: choose 1st heaviest, 2nd heaviest, ...

Problem 2

- ◆ Given a basket of apples, you want to choose a subset of them to total weight say 2 Kg. Basket has apples of weight 450 gm. And 400 gm.
- ◆ Greedy choice: $4 \times 450 = 1800$ gm.
- ◆ Alternate choice: $5 \times 400 = 2000$ gm.

Problem 3

- ◆ Subsequence problem

- ◆ Sequence - a, c, b, g, a, b, a, g, b, c, a

- ◆ Subsequence - a, c, b, g, a, b, a, g, b, c, a - (c, g, a, b, c)

Problem 3

- ◆ Subsequence problem
 - ◆ Given two sequences S_1 and S_2 , whether S_2 is a subsequence of S_1 .
 - ◆ $|S_1| = n, |S_2| = p$.

Subsequence problem

- ◆ Approach 1: search through all subsequences of length p and check if one of them is length p , and check if it is equal to S_2 .
- ◆ # of such sequences $= \binom{n}{p} \dots$

Subsequence problem

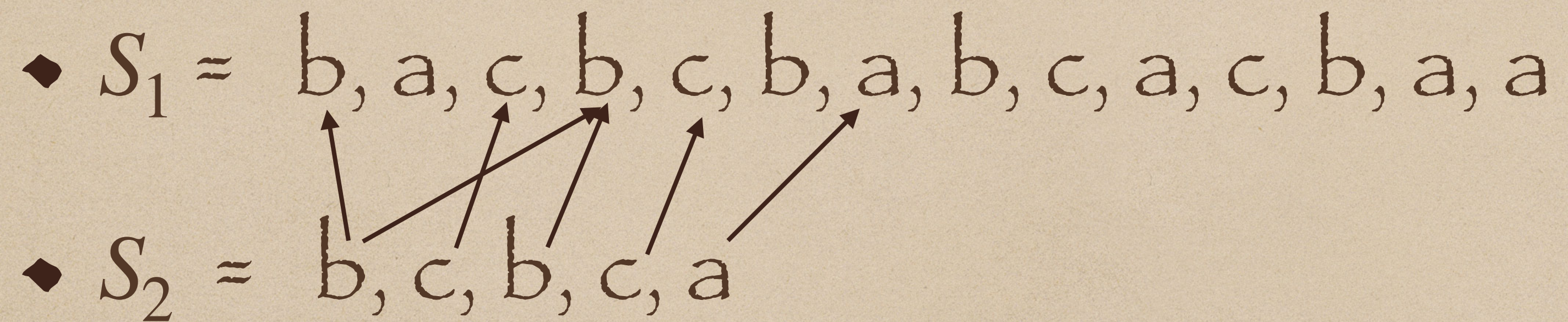
- ◆ Approach 2: Match letter by letter
 - ◆ $S_1 = b, a, c, b, c, b, a, b, c, a, c, b, a, a$
 - ◆ $S_2 = b, c, b, c, a$

Subsequence problem

- ◆ Approach 2: Match letter by letter

- ◆ $S_1 = b, a, c, b, c, b, a, b, c, a, c, b, a, a$

- ◆ $S_2 = b, c, b, c, a$



Subsequence problem

- ◆ Approach 2: Match letter by letter
 - ◆ Invariant: Match the current letter in S_2 with its first occurrence in S_1 , after the previous matching.

Subsequence problem

- ◆ Correctness
 - ◆ We need to argue that greedy approach works: If S_2 is indeed a subsequence of S_1 then, we should be able to get it by matching each subsequent letter of S_2 by matching with its first occurrence in S_1 , after the previous matching.

Subsequence problem

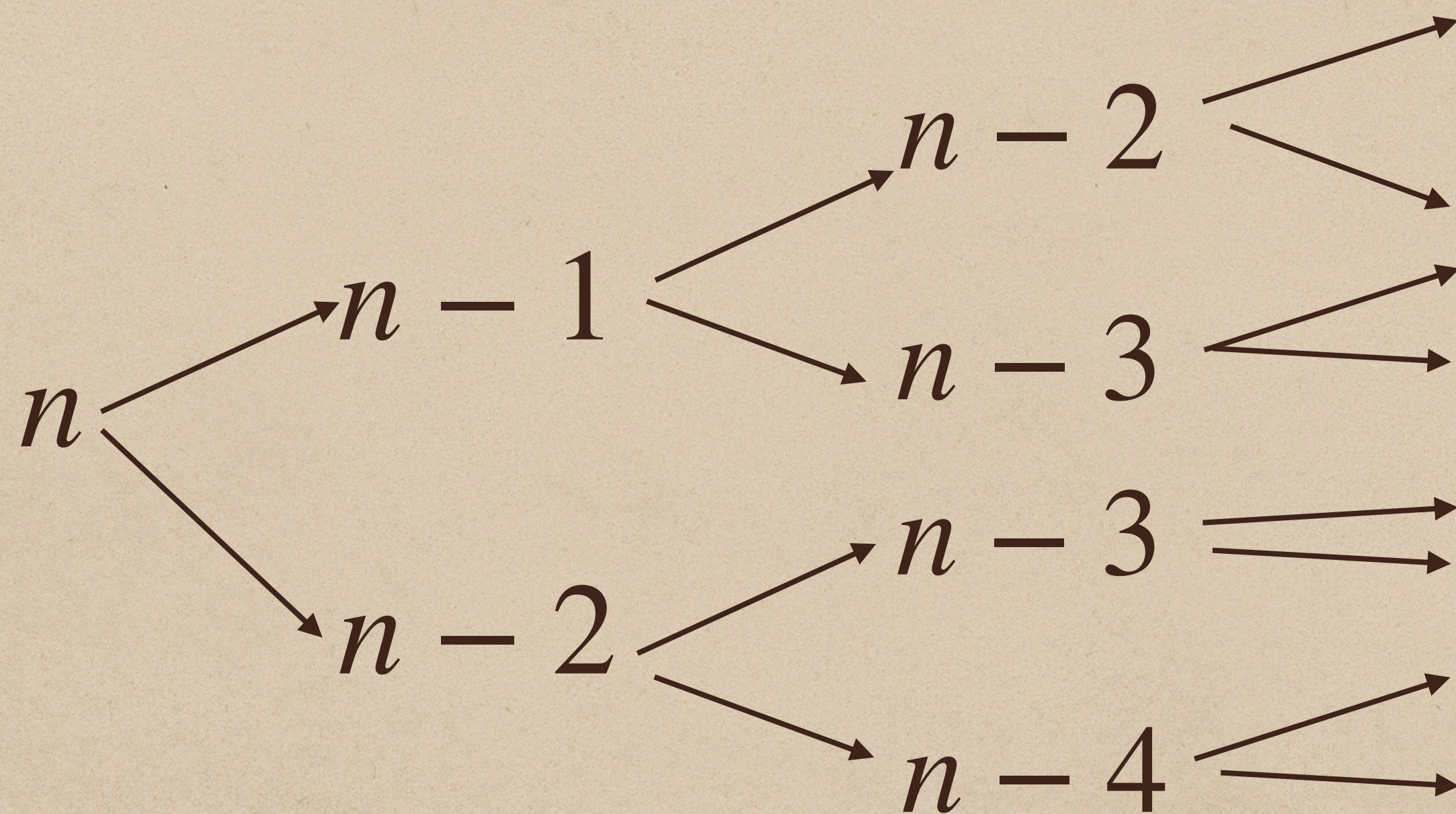
- ◆ Correctness

- ◆ Start with an arbitrary sequence of S_1 that matches with S_2 .
- ◆ Now, move the first letter of S_2 to its first occurrence in S_1 (its still a 'valid' subsequence!)
- ◆ Repeat this argument for every letter of S_2 .

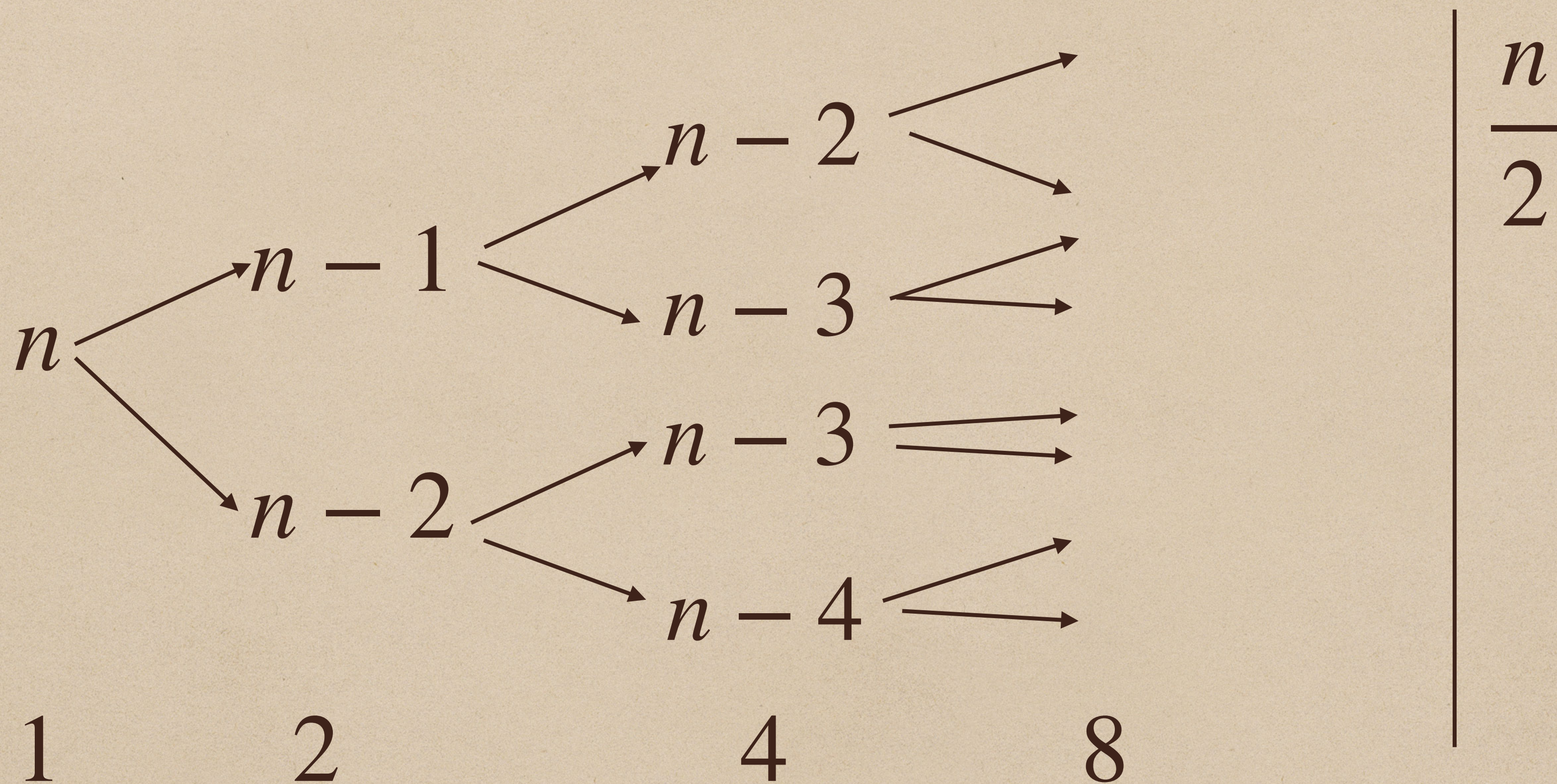
Dynamic Programming

- ◆ Simple example - $F_n = F_{n-1} + F_{n-2}$. $F_0 = F_1 = 1$
- ◆ Fibonacci (n) :
 - ◆ If $n = 0$, return 1.
 - ◆ If $n = 1$, return 1.
 - ◆ Else, return — Fibonacci (n) = Fibonacci (n-1) + Fibonacci (n-2).

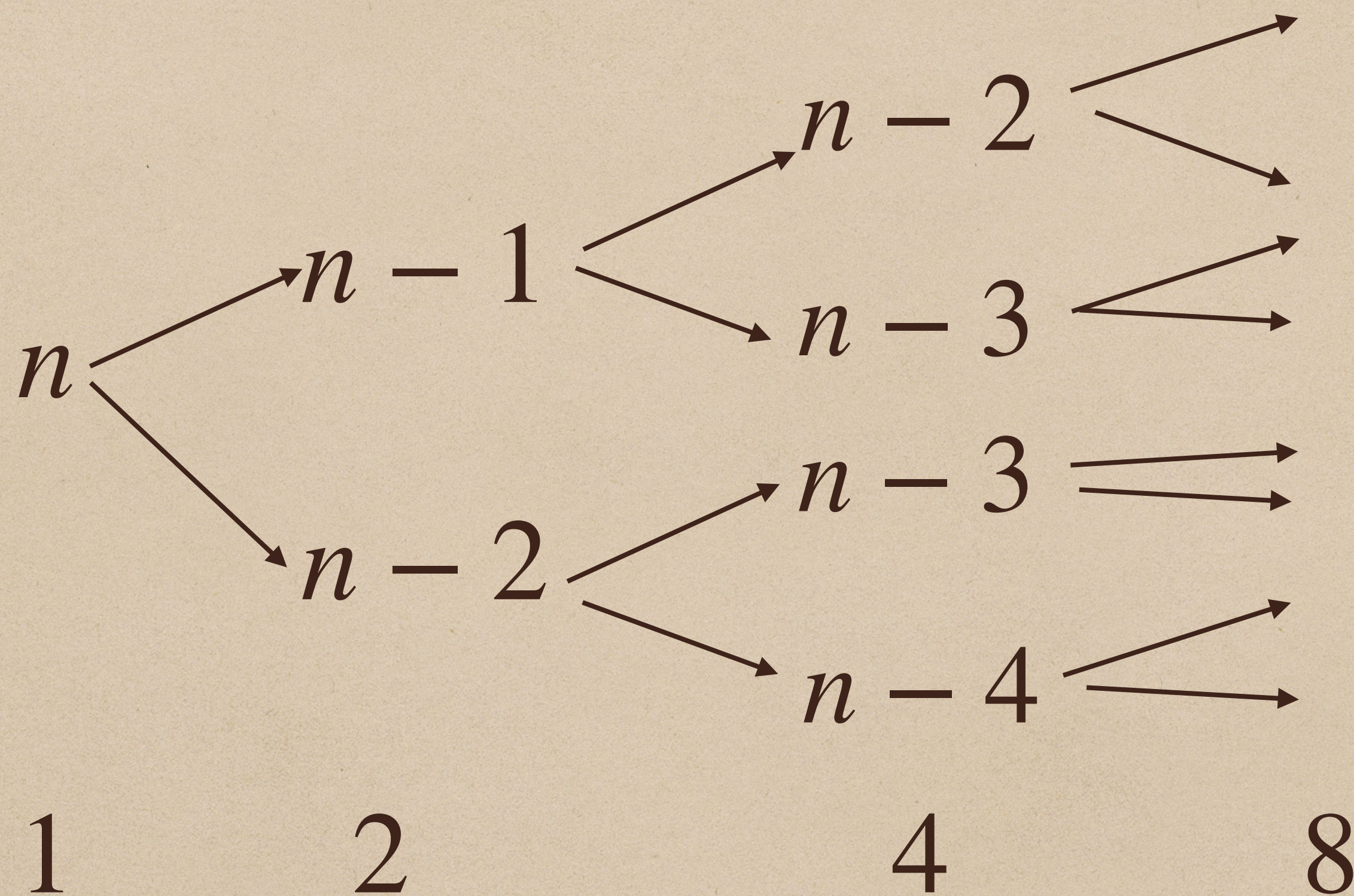
Dynamic Programming



Dynamic Programming



Dynamic Programming



$$\frac{n}{2}$$

At least $2^{\frac{n}{2}}$
recursive calls!

Bad
Implementation!

Dynamic Programming

- ◆ Better Implementation!
- ◆ Array F of length $n+1$
 - ◆ $F(0) = 1$
 - ◆ $F(1) = 1$

Dynamic Programming

Bottom Up

- ◆ Better Implementation!
- ◆ Array F of length $n+1$
 - ◆ $F(0) = 1$
 - ◆ $F(1) = 1$

For $i = 0$ to n

$$F(i) = \frac{F(i-1) + F(i-2)}$$

End

Already stored in an Array



Mantra of Dynamic Programming

- ◆ Reduce the Problem to many Subproblems.
- ◆ If the same subproblem instance appear multiple times, then
 - ◆ Solve it once, Store, and reuse [Memoization].

Mantra of Dynamic Programming

- ◆ Reduce the Problem to many Subproblems.
- ◆ If the same subproblem instance appear multiple times, then
 - ◆ Solve it once, Store, and reuse [Memoization].

If the total number of **Distinct Subproblem Instances** needed in the course of the Algorithm is **Polynomially bounded**, then we are good!